

Manual Técnico – ESTFlix

Projeto ESTFlix

Autor

Christian Garcia- 2025138837

Adam Afoulous- 2025157026

UC

Programação Web

Conteúdo

1. Arquitetura do Sistema.....	2
1.1 Visão Geral.....	2
1.2 Estrutura de Diretórios.....	2
1.3 Módulos de Sistema	3
1.3.1 Módulo de Autenticação e Utilizadores	3
1.3.2 Módulo de Perfis	3
1.3.3 Módulo de Conteúdos e Categorias Automáticas	3
1.4 Fluxo de Dados	4
2. Entidades e a sua Implementação.....	4
2.1 Descrição das Entidades	4
2.1.1 Utilizador (utilizadores).....	4
2.1.2 Perfil	4
2.1.3 Conteúdo	5
2.1.4 Favoritos	5
2.1.5 Histórico	5
2.2 API REST e Rotas.....	6
3. Descrição das Opções Tomadas	7
3.1 Decisões Tecnológicas	7
4. Limitações Técnicas e Desenvolvimento Futuro	7
4.1 Limitações Técnicas Atuais	7
4.2 Ideias para Desenvolvimento Futuro	7

1. Arquitetura do Sistema

1.1 Visão Geral

A aplicação segue uma arquitetura cliente-servidor com separação entre frontend e backend. O sistema corre de forma unificada: o servidor Express disponibiliza os ficheiros estáticos do frontend e expõe os pontos de acesso da API REST no mesmo domínio (localhost:3000). Os pedidos assíncronos são realizados via AJAX, atualizando a interface sem recarregar a página.

1.2 Estrutura de Diretórios

backend/

|- package.json Dependências (express, mysql2, cors)

├ server.js Servidor unificado, configuração de rotas e ligação à BD

|- public/ Pasta de distribuição do Frontend

├── index.html Página de Login, Registo e Escolha de Perfis

├── catalogo.html Página principal de consumo de conteúdos

├── admin.html Painel de gestão administrativa de filmes

├── css/

 |- style.css Folha de estilos globais e regras de responsividade

└─ js/

 ├── main.js Lógica de utilizadores, criação, remoção e limites de perfis

 ├── catalogo.js Injeção dinâmica de filmes e carrosséis de categorias

 ├── admin.js Manipulação do CRUD de conteúdos (Adicionar, Editar, Apagar)

 └─ storage.js Classes utilitárias de autenticação e estados locais

1.3 Módulos de Sistema

1.3.1 Módulo de Autenticação e Utilizadores

Responsável por gerir o acesso e a criação de contas na plataforma.

Rotas API: /api/login (POST), /api/registo (POST).

Armazenamento e validação de credenciais diretamente na base de dados.

1.3.2 Módulo de Perfis

Permite que cada conta de utilizador efetue a gestão de identidades independentes.

Rotas API: /api/perfis (GET, POST), /api/perfis/:id (DELETE).

Regras de Negócio: Restrição rigorosa de um limite máximo de 5 perfis por utilizador

1.3.3 Módulo de Conteúdos e Categorias Automáticas

Núcleo do sistema que gere o catálogo e a organização visual dos filmes.

- Rotas API: /api/conteudos (GET, POST), /api/conteudos/:id (PUT, DELETE).
- Mapeamento Automático: O catálogo lê o campo de texto genero de cada filme e extrai dinamicamente os valores únicos. Se existirem filmes para um género, a linha correspondente é injetada na interface; se não existirem, a categoria não é exibida.

1.3.4 Módulo de Favoritos e Histórico

Regista as interações de cada perfil com os filmes do catálogo.

- Rotas API: /api/favoritos (GET, POST, DELETE), /api/historico (POST), /api/historico/:perfilId (GET).

1.4 Fluxo de Dados

1. O utilizador interage com o ecrã (ex: clica num filme para abrir o detalhe).
2. O frontend despacha um pedido fetch assíncrono com rotas relativas para a API.
3. O ficheiro server.js recebe o pedido, extrai os parâmetros e executa comandos SQL na base de dados MySQL.
4. O MySQL processa os dados e devolve o resultado ao servidor Node.js.
5. O servidor formata a resposta em formato JSON e envia-a de volta para o cliente.
6. O JavaScript do frontend interpreta o JSON e reconstrói o DOM da página sem recarregamento total.

2. Entidades e a sua Implementação

2.1 Descrição das Entidades

2.1.1 Utilizador (utilizadores)

Representa as contas globais de acesso à plataforma.

- id: INT (AUTO_INCREMENT, Chave Primária)
- nome: VARCHAR(100)
- email: VARCHAR(150) (Único)
- password: VARCHAR(255)

2.1.2 Perfil

Sub-contas criadas por um utilizador para segmentar a sua experiência.

- id: INT (AUTO_INCREMENT, Chave Primária)
- nome: VARCHAR(100)
- foto: VARCHAR(500)
- utilizador_id: INT (Chave Estrangeira ligada a utilizadores com ON DELETE CASCADE)

2.1.3 Conteúdo

Os filmes e metadados registados na aplicação.

- id: INT (AUTO_INCREMENT, Chave Primária)
- titulo: VARCHAR(150) (Único)
- descricao: TEXT
- genero: VARCHAR(100)
- ano: INT
- classificacao: DECIMAL(2,1)
- imageUrl: VARCHAR(500)

2.1.4 Favoritos

Tabela associativa muitos-para-muitos para os filmes guardados pelo perfil.

- perfil_id: INT (Chave Primária Composta, Estrangeira ligada a perfis com ON DELETE CASCADE)
- conteudo_id: INT (Chave Primária Composta, Estrangeira ligada a conteudos com ON DELETE CASCADE)

2.1.5 Histórico

Registo cronológico de visualizações de filmes.

- id: INT (AUTO_INCREMENT, Chave Primária)
- perfil_id: INT (Chave Estrangeira ligada a perfis com ON DELETE CASCADE)
- conteudo_id: INT (Chave Estrangeira ligada a conteudos com ON DELETE CASCADE)
- data_visualizacao: DATETIME (DEFAULT CURRENT_TIMESTAMP)

2.2 API REST e Rotas

Método	Rota	Descrição
POST	/api/login	Autentica um utilizador existente
POST	/api/registo	Cria uma nova conta na base de dados
GET	/api/perfis	Obtém todos os perfis do sistema
POST	/api/perfis	Cria um novo perfil para o utilizador logado
DELETE	/api/perfis/:id	Elimina um perfil específico e dependências
GET	/api/conteudos	Lista todos os filmes disponíveis
POST	/api/conteudos	Adiciona um novo filme ao catálogo (Admin)
PUT	/api/conteudos/:id	Atualiza os metadados de um filme (Admin)
DELETE	/api/conteudos/:id	Remove um filme do catálogo (Admin)
GET	/api/favoritos/:perfilId	Lista os favoritos de um perfil específico
POST	/api/favoritos	Associa um filme aos favoritos de um perfil

Método	Rota	Descrição
DELETE	/api/favoritos	Remove uma associação de favorito
GET	/api/historico/:perfilId	Obtém o histórico otimizado e sem duplicados
POST	/api/historico	Adiciona uma entrada de visualização ao histórico

3. Descrição das Opções Tomadas

3.1 Decisões Tecnológicas

Optou-se por utilizar o Express não só como API de dados, mas também como servidor estático (express.static). Isto eliminou por completo erros de CORS e conflitos de portas, centralizando toda a aplicação em localhost:3000.

4. Limitações Técnicas e Desenvolvimento Futuro

4.1 Limitações Técnicas Atuais

Falta de Encriptação de Passwords: As passwords dos utilizadores estão a ser gravadas e validadas em texto limpo na base de dados, sem o uso de hashing

Falta de Descrição: Os filmes não possuem uma descrição.

4.2 Ideias para Desenvolvimento Futuro

Implementação de Encriptação: Integração de segurança criptográfica no processo de criação de conta e login.

Implementação da Descrição: Permitir inserir uma descrição a um filme.